# Robótica del Servicio
# ING 01335

Facultad de Ingeniería
Politécnico Colombiano Jaime Isaza Cadavid

Deimer Miranda Montoya, MSc.(c).
deimer_miranda91162@elpoli.edu.co

# Clase 4: Fundamentos de ROS 2 Humble

## Del modelo matemático al mensaje robótico



Deimer Miranda Montoya, MSc.(c).

# Propósito de la clase

Al finalizar esta sesión, el estudiante estará en capacidad de:

- reconocer la función de ROS 2 dentro de un sistema robótico;
- diferenciar nodos, tópicos y mensajes;
- ejecutar comandos básicos de ROS 2 desde terminal;
- inspeccionar nodos y tópicos activos;
- controlar Turtlesim mediante mensajes de velocidad;
- observar la pose publicada por el simulador;
- relacionar velocidad lineal, velocidad angular y movimiento.

**Idea clave**

ROS 2 no reemplaza el modelo matemático del robot. ROS 2 permite implementar, comunicar y observar los componentes que usan ese modelo.

# Conexión matemática

## Posición, orientación y movimiento



Deimer Miranda Montoya, MSc.(c).

# ¿Qué sabemos hasta ahora?

En las clases anteriores trabajamos:

- puntos y vectores en el plano;
- distancia euclidiana;
- dirección mediante $\operatorname{atan2}$;
- coordenadas cartesianas y polares;
- matriz de rotación $R(\theta)$;
- sistemas de referencia local y global.

**Resultado**

Ahora comenzaremos a representar computacionalmente estas ideas usando ROS 2 y Turtlesim.

# La idea central de la sesión

Modelo matemático $\rightarrow$ Mensaje $\rightarrow$ Tópico $\rightarrow$ Movimiento




**Idea clave**

El movimiento de un robot puede comenzar como una ecuación, pero en ROS 2 debe expresarse como un mensaje que viaja por un tópico.

# ¿Qué es ROS 2?

## Infraestructura para sistemas robóticos distribuidos

# ¿Qué problema ayuda a resolver ROS 2?

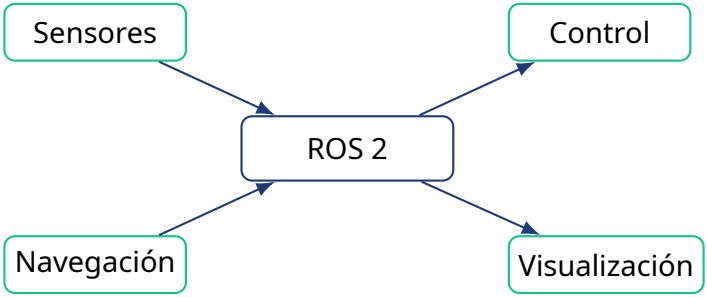
Un robot real no suele ser un único programa.
Normalmente está formado por varios componentes:

- sensores;
- actuadores;
- algoritmos de control;
- estimación de posición;
- navegación;
- visualización;
- registro de datos.

**Idea clave**

ROS 2 permite que estos componentes se comuniquen de forma organizada.

# Robot como sistema distribuido




## ROS 2

ROS 2 es una infraestructura de software que permite construir sistemas robóticos a partir de procesos que se comunican mediante mensajes.

# ROS 2 no es el robot

ROS 2 no es:

- un robot físico;
- un simulador por sí solo;
- un lenguaje de programación;
- un controlador automático mágico.

ROS 2 es una infraestructura para:

- ejecutar procesos;
- comunicar módulos;
- transportar datos;
- observar el estado del sistema;
- organizar software robótico.

# De la matemática al sistema

Por ejemplo, en un robot móvil podemos tener:

$$v$$

como velocidad lineal y:

$$\omega$$

como velocidad angular.
Pero ROS 2 necesita enviar estas cantidades dentro de un mensaje.

$$(v, \omega) \rightarrow \texttt{Twist} \rightarrow \texttt{/cmd_vel} \rightarrow \text{movimiento}$$

**Resultado**

La matemática define qué queremos hacer. ROS 2 define cómo lo comunicamos entre componentes.

# Conceptos fundamentales

## Nodo, tópico y mensaje

# Nodo

Un nodo es un proceso que ejecuta una tarea específica dentro de ROS 2.
En lugar de construir un único programa grande para todo el robot, ROS 2 permite dividir el sistema en varios nodos especializados.

## Nodo

Un nodo es una unidad de ejecución. Cada nodo se encarga de una parte concreta del sistema robótico y puede comunicarse con otros nodos.

# Nodo: idea robótica

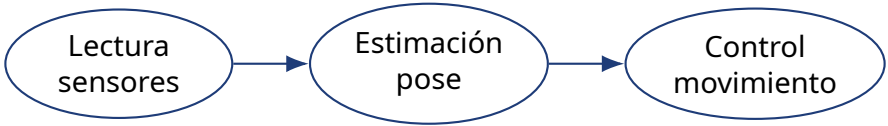
En un robot móvil podríamos tener:

- un nodo que lee sensores de distancia;
- un nodo que estima la posición del robot;
- un nodo que calcula una acción de control;
- un nodo que envía velocidades a los motores;
- un nodo que registra datos para análisis posterior.

**Idea clave**

Cada nodo tiene una responsabilidad clara. Esto facilita probar, corregir y reemplazar partes del sistema sin modificar todo el robot.

# Nodo: separación de responsabilidades




**Resultado**

Un sistema robótico se vuelve más organizado cuando cada nodo cumple una función específica.

# Tópico

Un tópico es un canal de comunicación por donde viajan mensajes.
Los nodos pueden:

- publicar información en un tópico;
- suscribirse a un tópico para recibir información.

## Tópico

Un tópico no ejecuta cálculos. Su función es transportar mensajes entre nodos que producen información y nodos que la necesitan.

# Tópico: analogía

Puede pensarse en un tópico como un canal con un nombre.
Por ejemplo:

`/turtle1/cmd_vel`

es el canal por donde se envían comandos de velocidad.

`/turtle1/pose`

es el canal por donde Turtlesim publica posición y orientación.

**Idea clave**

En ROS 2 no basta con tener datos. Es necesario saber en qué tópico se publican y qué tipo de mensaje transportan.

# Comunicación por tópicos



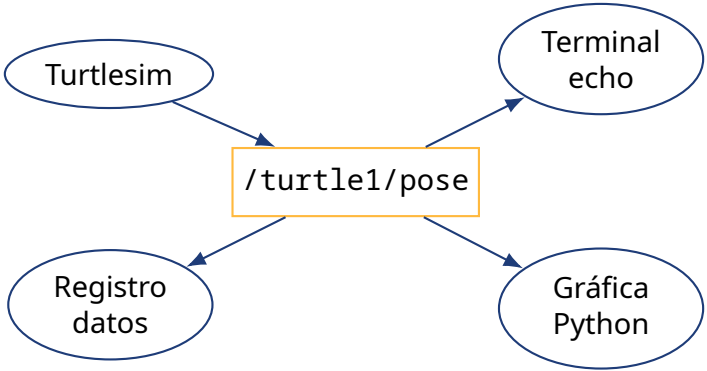

**Idea clave**

El publicador no necesita saber quién recibe el mensaje. El suscriptor solo necesita conocer el tópico al cual escuchar.

# Varios nodos pueden usar el mismo tópico

Un mismo tópico puede tener:

- varios publicadores;
- varios suscriptores;
- un publicador y múltiples consumidores de información.



# Mensaje

Un mensaje define la estructura de los datos que viajan por un tópico.
No todos los tópicos transportan el mismo tipo de información.

Por ejemplo:

`/turtle1/cmd_vel`

transporta un mensaje de velocidad.

`/turtle1/pose`

transporta un mensaje de posición y orientación.

## Mensaje

Un mensaje es el formato de datos que ROS 2 utiliza para comunicar nodos. Define qué campos existen y qué tipo de dato tiene cada campo.

# Mensaje: estructura

Un mensaje no es solo un número. Puede contener varios campos.
Por ejemplo, un mensaje de velocidad tipo `Twist` contiene:

`linear` y `angular`

Cada parte puede tener componentes en:

$$x, \quad y, \quad z$$

**Idea clave**

Para mover Turtlesim en el plano usaremos principalmente `linear.x` y `angular.z`.

# Mensaje: relación con la matemática

En robótica móvil plana usamos:

$$v$$

para velocidad lineal, y:

$$\omega$$

para velocidad angular.
En ROS 2 estas variables se escriben dentro del mensaje:

$$v \rightarrow \texttt{linear.x}$$

$$\omega \rightarrow \texttt{angular.z}$$

**Resultado**

La matemática define la variable. El mensaje define cómo esa variable viaja dentro del sistema ROS 2.

# Publicador

Un publicador es la parte de un nodo que envía mensajes a un tópico.
Ejemplo:

`/teleop_turtle`

publica comandos de velocidad en:

`/turtle1/cmd_vel`

**Publicador**

Un publicador produce mensajes y los coloca en un tópico específico.

# Suscriptor

Un suscriptor es la parte de un nodo que recibe mensajes desde un tópico.
Ejemplo:

`/turtlesim`

se suscribe al tópico:

`/turtle1/cmd_vel`

para recibir velocidades y mover la tortuga.

**Suscriptor**

Un suscriptor escucha un tópico y ejecuta una acción cuando recibe un mensaje.

# Publicador y suscriptor

**Publicador**

- genera datos;
- los envía a un tópico;
- no controla directamente quién los recibe.

`publish()`

**Suscriptor**

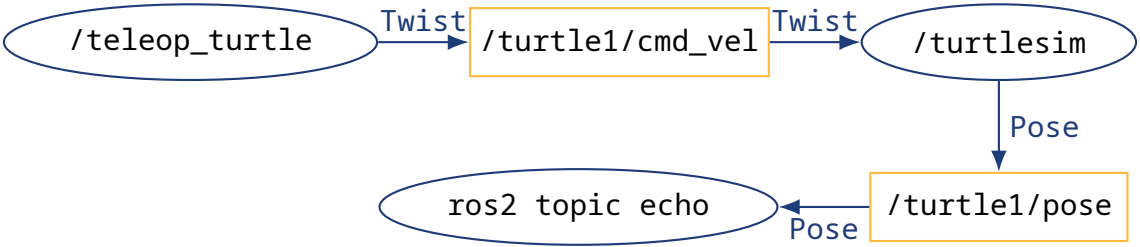
- escucha un tópico;
- recibe mensajes;
- procesa la información recibida.

`subscribe()`

**Resultado**

Este patrón será la base de casi todo el trabajo con ROS 2.

# Ejemplo completo en Turtlesim



# Lectura del diagrama

En el diagrama anterior:

- `/teleop_turtle` es un nodo;
- `/turtle1/cmd_vel` es un tópico;
- el mensaje que viaja allí es `Twist`;
- `/turtlesim` recibe ese mensaje;
- `/turtlesim` publica la pose en `/turtle1/pose`;
- la terminal puede suscribirse para observar esa pose.

**Idea clave**

En ROS 2 siempre debemos preguntarnos: ¿quién publica?, ¿en qué tópico?, ¿qué tipo de mensaje viaja?, ¿quién se suscribe?

# Exploración de mensajes ROS 2

## De un dato simple a una estructura de velocidad

# Antes de usar Turtlesim

Ya sabemos que un mensaje define la estructura de los datos que viajan entre nodos. Pero antes de utilizar mensajes en un robot, podemos explorarlos directamente desde la terminal.

Comenzaremos con estructuras simples:

$$\texttt{Int32} \rightarrow \texttt{Float64}$$

y progresivamente llegaremos a:

$$\texttt{Vector3} \rightarrow \boxed{\texttt{Twist}}$$

**Idea clave**

La complejidad de un mensaje depende de la información que necesitamos representar.

# Nombre completo de un tipo de mensaje

En ROS 2 encontraremos nombres como:

`std_msgs/msg/Int32`

Este nombre contiene tres partes:




## Notación

La expresión `paquete/msg/Mensaje` identifica de manera completa un tipo de mensaje dentro de ROS 2.

# ¿Qué es `std_msgs`?

El paquete:

`std_msgs`

contiene mensajes básicos utilizados para transportar valores sencillos.
Por ejemplo:

- números enteros;
- números reales;
- valores booleanos;
- cadenas de caracteres.

**Idea clave**

Aunque estos mensajes son simples, permiten comprender cómo ROS 2 encapsula un dato antes de enviarlo.

# Explorar las interfaces disponibles

Podemos consultar las interfaces conocidas por ROS 2:

```
ros2 interface list
```

La lista puede ser extensa.
Para buscar mensajes relacionados con `std_msgs`:

```
ros2 interface list | grep std_msgs
```

**ros2 interface list**

Muestra las interfaces disponibles en la instalación actual de ROS 2.

# Explorar un paquete de interfaces

También podemos consultar las interfaces pertenecientes a un paquete:

```
ros2 interface package std_msgs
```

Entre los resultados podremos encontrar mensajes como:

```
std_msgs/msg/Bool
std_msgs/msg/Float64
std_msgs/msg/Int32
std_msgs/msg/String
```

**Idea clave**

Un paquete puede contener varios tipos de mensajes diseñados para representar diferentes clases de información.

# Mensajes simples

## Un dato también necesita una estructura

# Mensaje Int32

Consultemos:

```
ros2 interface show std_msgs/msg/Int32
```

La estructura es:

```
int32 data
```

Esto significa que el mensaje contiene un campo llamado:

`data`

de tipo:

`int32`

# Interpretación de Int32

Podemos imaginar el mensaje como un pequeño contenedor:

```
std_msgs/msg/Int32
  data : int32
```

Por ejemplo:

$$\texttt{data} = 25$$

**Idea clave**

El valor 25 no viaja “solo”. Viaja dentro de una estructura definida por el mensaje `Int32`.

# Tipo del campo y tipo del mensaje

Es importante diferenciar:

`int32`

de:

`std_msgs/msg/Int32`

El primero es un:

tipo de dato

El segundo es un:

tipo de mensaje ROS 2

**Atención**

`int32` no es un tópico ni un mensaje completo. Es el tipo de dato utilizado por el campo `data`.

# Mensaje Float64

Ahora consulte:

```
ros2 interface show std_msgs/msg/Float64
```

La estructura es:

```
float64 data
```

Un posible contenido sería:

$$\texttt{data} = 3.1416$$

**Float64**

`std_msgs/msg/Float64` permite transportar un número real de precisión doble dentro del campo `data`.

# Ejemplo robótico con Float64

Supongamos que un sensor mide una distancia:

$$d = 0.73 \text{ m}$$

El valor matemático es:

$$0.73$$

Una posible representación mediante un mensaje simple sería:

```
data: 0.73
```

```
Float64
data = 0.73
```

# Pero un robot necesita estructuras más grandes

Un único número puede representar:

$$d \rightarrow v \rightarrow \theta$$

Pero algunas cantidades necesitan varias componentes.
Por ejemplo, un vector tridimensional:

$$\mathbf{v} = \begin{bmatrix} v_x, v_y, v_z \end{bmatrix}^T$$

requiere almacenar tres valores.

**Idea clave**

Cuando la información tiene varias componentes, necesitamos mensajes con estructuras más complejas.

# Mensajes compuestos

Cuando un solo campo ya no es suficiente

# Explorar Vector3

Consultemos:

```
ros2 interface show geometry_msgs/msg/Vector3
```

Su estructura es:

```
float64 x
float64 y
float64 z
```

Ahora el mensaje contiene:

$$3$$

campos independientes.

# Interpretación de Vector3

Podemos representarlo como:

$$\mathbf{v} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

En ROS 2:

```
geometry_msgs/msg/Vector3
    x : float64
    y : float64
    z : float64
```

# Ejemplo de Vector3

Supongamos:

$$\mathbf{v} = \begin{bmatrix} 2.0 \\ -1.5 \\ 0.4 \end{bmatrix}$$

El mensaje contendría:

```
x: 2.0

y: -1.5

z: 0.4
```

**Resultado**

El mensaje `Vector3` permite agrupar tres componentes relacionadas dentro de una sola estructura.

# La estructura puede contener otras estructuras

Hasta ahora tenemos:

`Int32` $\rightarrow$ un campo

`Float64` $\rightarrow$ un campo

`Vector3` $\rightarrow$ tres campos

Pero ROS 2 también permite que un mensaje contenga:

otros mensajes

**Idea clave**

Esta composición permite construir estructuras de datos progresivamente más completas.

# Mensaje Twist

## Dos vectores dentro de un mensaje

# Explorar Twist desde terminal

Ejecute:

```
ros2 interface show geometry_msgs/msg/Twist
```

Su estructura conceptual es:

```
Vector3 linear
Vector3 angular
```

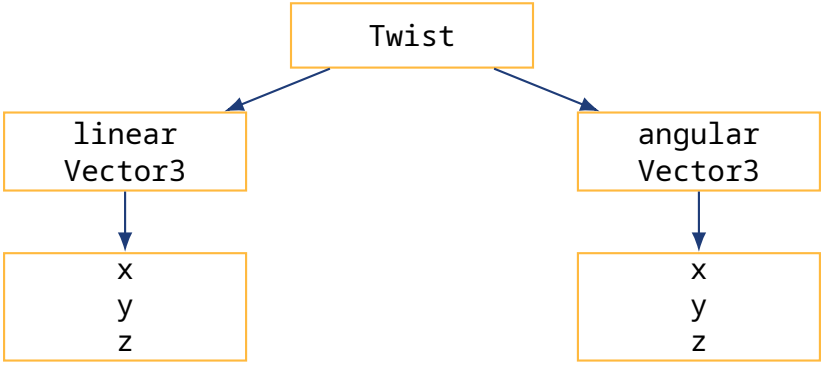
Esto significa que `Twist` contiene:

$$2$$

mensajes de tipo:

`Vector3`

# Construcción jerárquica de Twist



# Estructura completa de Twist

Podemos pensar en el mensaje como:

$$\texttt{Twist} = \begin{cases} \texttt{linear} = \begin{cases} x \\ y \\ z \end{cases} \\ \texttt{angular} = \begin{cases} x \\ y \\ z \end{cases} \end{cases}$$

Por tanto, existen seis componentes numéricas:

$$v_x,\ v_y,\ v_z,\ \omega_x,\ \omega_y,\ \omega_z$$

# Interpretación física

El bloque:

```
linear
```

representa velocidad lineal:

$$\mathbf{v} = \begin{bmatrix} v_x \\ v_y \\ v_z \end{bmatrix}$$

Mientras que:

```
angular
```

representa velocidad angular:

$$\boldsymbol{\omega} = \begin{bmatrix} \omega_x \\ \omega_y \\ \omega_z \end{bmatrix}$$

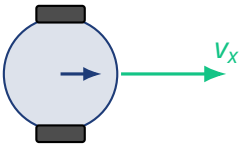
# ¿Por qué usamos `linear.x`?

Para un robot móvil que se desplaza sobre un plano, consideraremos:

$$X$$

como la dirección hacia adelante.
Por tanto:

$$\boxed{v = \texttt{linear.x}}$$



# ¿Por qué usamos `angular.z`?

El robot se mueve sobre el plano:

$$XY$$

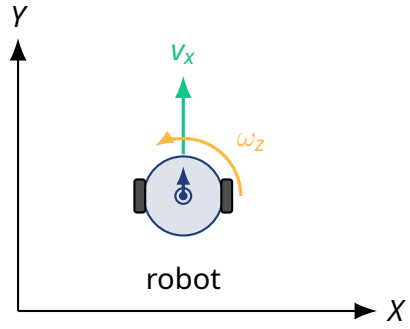
La rotación que cambia su orientación ocurre alrededor del eje:

$$Z$$

que es perpendicular al plano.
Por tanto:

$$\boxed{\omega = \texttt{angular.z}}$$

# Velocidad angular alrededor de Z




## Idea clave

Aunque el movimiento ocurre en el plano *XY*, la rotación se describe alrededor del eje perpendicular *Z*.

# Del mensaje al movimiento planar

Para nuestro caso:

$$\begin{aligned} \texttt{linear.x} &= v \\ \texttt{linear.y} &= 0 \\ \texttt{linear.z} &= 0 \end{aligned}$$

y:

$$\begin{aligned} \texttt{angular.x} &= 0 \\ \texttt{angular.y} &= 0 \\ \texttt{angular.z} &= \omega \end{aligned}$$

Por tanto:

$$\boxed{\mathbf{u} = \begin{bmatrix} v \\ \omega \end{bmatrix}}$$

# De la matemática a Twist




**Resultado**

Ahora el mensaje `Twist` deja de ser una estructura abstracta: es una representación computacional de variables físicas de movimiento.

# Exploración guiada

## Leer una interfaz antes de utilizarla

# Secuencia de exploración

Ejecute en orden:

```
ros2 interface show std_msgs/msg/Int32
```

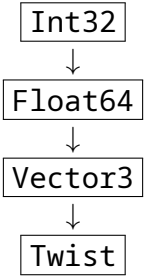
```
ros2 interface show std_msgs/msg/Float64
```

```
ros2 interface show geometry_msgs/msg/Vector3
```

```
ros2 interface show geometry_msgs/msg/Twist
```

# Compare las estructuras

Complete mentalmente la progresión:




## Idea clave

No estamos cambiando simplemente de tipo numérico. Estamos pasando de estructuras simples a estructuras compuestas capaces de representar información física más rica.

# Preguntas antes de continuar

Analice:

1. ¿Cuál es la diferencia entre `float64` y `std_msgs/msg/Float64`?
2. ¿Cuántos campos posee `Vector3`?
3. ¿Qué tipo de mensaje son `linear` y `angular` dentro de `Twist`?
4. ¿Cuántos valores numéricos contiene finalmente un `Twist`?
5. ¿Por qué para movimiento planar utilizaremos principalmente `linear.x` y `angular.z`?

# Estamos listos para utilizar Turtlesim

Ahora sabemos interpretar:

`geometry_msgs/msg/Twist`

y entendemos que contiene:

$v \rightarrow$ `linear.x`

$\omega \rightarrow$ `angular.z`

**Resultado**

El siguiente paso será identificar qué nodo publica este mensaje, por qué tópico viaja y qué nodo lo recibe para producir movimiento.

# Turtlesim

## Un laboratorio mínimo para entender ROS 2

# ¿Por qué usar Turtlesim?

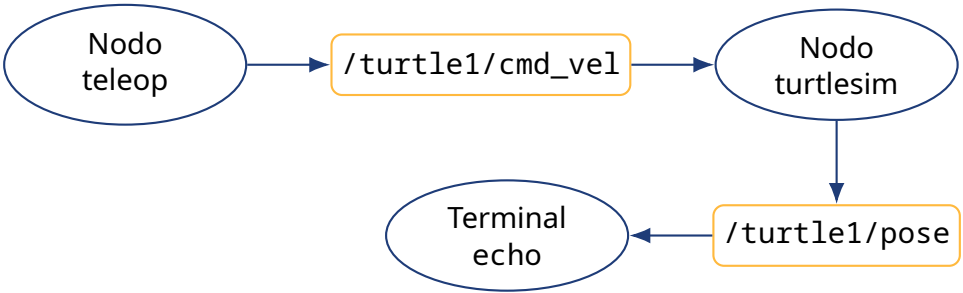
Turtlesim permite estudiar ROS 2 sin depender todavía del robot físico.
Con Turtlesim podemos practicar:

- ejecución de nodos;
- publicación de mensajes;
- suscripción a tópicos;
- comandos de inspección;
- movimiento en el plano;
- interpretación de pose.

**Idea clave**

Turtlesim es simple, pero contiene las ideas esenciales que luego veremos en un robot móvil real.

# Arquitectura básica con Turtlesim



# Movimiento en Turtlesim

Turtlesim se mueve a partir de comandos de velocidad.
La tortuga recibe mensajes en:

`/turtle1/cmd_vel`

y publica su pose en:

`/turtle1/pose`

velocidad $\rightarrow$ movimiento $\rightarrow$ pose

**Resultado**

Esta relación es la primera conexión práctica entre modelo de movimiento y comunicación ROS 2.

# Preparar la terminal

## Cargar ROS 2 Humble

# Verificar ROS 2

Abra una terminal y verifique la distribución activa:

```
echo $ROS_DISTRO
```

La salida esperada es:

```
humble
```

**Atención**

Si no aparece `humble`, debe cargar el entorno de ROS 2.

# Cargar ROS 2 Humble

Ejecute:

```
source /opt/ros/humble/setup.bash
```

Verifique nuevamente:

```
echo $ROS_DISTRO
```

La salida debe ser:

```
humble
```

**Idea clave**

El comando `source` carga las variables necesarias para que la terminal reconozca los comandos y paquetes de ROS 2.

# Ejecutar Turtlesim

## Primeros nodos ROS 2

# Instalar Turtlesim

Si Turtlesim no se encuentra instalado, ejecute:

```
sudo apt update
sudo apt install -y ros-humble-turtlesim
```

**Resultado**

Después de instalarlo, podremos ejecutar el nodo principal del simulador.

# Ejecutar el simulador

En una terminal, ejecute:

```
ros2 run turtlesim turtlesim_node
```

Deberá aparecer una ventana con una tortuga.

## ros2 run

El comando `ros2 run` ejecuta un programa perteneciente a un paquete de ROS 2.

# Mover la tortuga con teclado

Abra una segunda terminal y ejecute:

```
ros2 run turtlesim turtle_teleop_key
```

Use las flechas del teclado para mover la tortuga.

**Atención**

La terminal donde se ejecuta `turtle_teleop_key` debe estar activa para que el teclado controle la tortuga.

# ¿Qué está ocurriendo?

Cuando presionamos una tecla:

1. el nodo de teleoperación genera un mensaje de velocidad;
2. el mensaje se publica en `/turtle1/cmd_vel`;
3. el nodo de Turtlesim recibe el mensaje;
4. la tortuga modifica su movimiento;
5. Turtlesim publica la nueva pose.

**Resultado**

Ya tenemos un sistema con nodos, tópicos y mensajes funcionando.

# Inspección de nodos

Observar qué procesos están activos

# Listar nodos

Con Turtlesim y teleop en ejecución, abra otra terminal y ejecute:

```
ros2 node list
```

Una salida posible es:

```
/teleop_turtle
/turtlesim
```

**ros2 node list**

Muestra los nodos activos que actualmente participan en el sistema ROS 2.

# Interpretación de los nodos

`/turtlesim`

representa el nodo del simulador.

`/teleop_turtle`

representa el nodo que lee el teclado y publica comandos de movimiento.



# Inspección de tópicos

Observar por dónde viajan los datos

# Listar tópicos

Ejecute:

```
ros2 topic list
```

Entre los tópicos disponibles deberían aparecer:

```
/turtle1/cmd_vel
/turtle1/pose
```

**ros2 topic list**

Muestra los canales de comunicación disponibles en el sistema ROS 2.

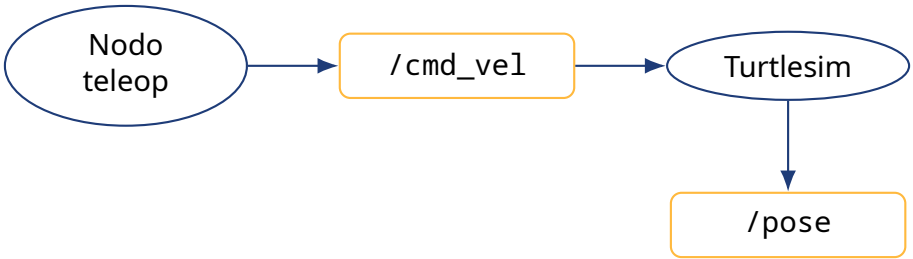
# Dos tópicos importantes

`/turtle1/cmd_vel`

recibe comandos de velocidad.

`/turtle1/pose`

publica la posición y orientación de la tortuga.



# Información de un tópico

Para conocer el tipo de mensaje de un tópico, ejecute:

```
ros2 topic info /turtle1/cmd_vel
```

Una salida posible es:

```
Type: geometry_msgs/msg/Twist
Publisher count: 1
Subscription count: 1
```

**Resultado**

El tópico `/turtle1/cmd_vel` transporta mensajes de tipo `Twist`.

# Mensaje Twist

## Velocidad lineal y velocidad angular

# Estructura conceptual de Twist

Un mensaje `Twist` contiene dos partes principales:

`linear`

y:

`angular`

`linear.x` $\rightarrow$ velocidad lineal hacia adelante

`angular.z` $\rightarrow$ velocidad angular de giro

**Idea clave**

En Turtlesim trabajaremos principalmente con `linear.x` y `angular.z`.

# Relación con el modelo matemático

En robótica móvil plana solemos usar:

$$v$$

para la velocidad lineal y:

$$\omega$$

para la velocidad angular.
En ROS 2, para Turtlesim:

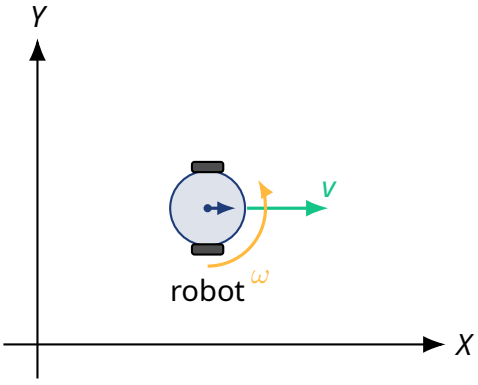
$$v \longrightarrow \texttt{linear.x}$$

$$\omega \longrightarrow \texttt{angular.z}$$

**Resultado**

La variable matemática se convierte en un campo dentro de un mensaje.

# Interpretación del movimiento




**Idea clave**

La velocidad lineal desplaza la tortuga. La velocidad angular cambia su orientación.

# Publicar mensajes

## Mover Turtlesim sin teclado

# Publicar una velocidad

Podemos enviar un mensaje directamente al tópico:

```
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 2.0}, angular: {z: 0.0}}"
```

Esto publica una velocidad lineal hacia adelante.

**Resultado**

Si `angular.z` es cero, la tortuga avanza sin girar.

# Publicar con frecuencia limitada

Para publicar un mensaje una sola vez:

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 2.0}, angular: {z: 0.0}}"
```

Para publicar a una frecuencia determinada:

```
ros2 topic pub -r 2 /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 1.0}, angular: {z: 0.5}}"
```

**Atención**

Un comando con `-r` sigue publicando hasta detenerlo con `Ctrl + C`.

# Efecto de los comandos

| `linear.x` | `angular.z` | **Efecto** |
|---|---|---|
| $> 0$ | $0$ | avanza recto |
| $0$ | $> 0$ | gira antihorario |
| $0$ | $< 0$ | gira horario |
| $> 0$ | $> 0$ | curva antihoraria |
| $> 0$ | $< 0$ | curva horaria |

**Idea clave**

El movimiento de la tortuga permite observar directamente la relación entre velocidad lineal y velocidad angular.

# Observar la pose

Posición y orientación publicadas por Turtlesim

# Escuchar el tópico de pose

Ejecute:

```
ros2 topic echo /turtle1/pose
```

Mientras la tortuga se mueve, observe cómo cambian:

- x;
- y;
- `theta`;
- `linear_velocity`;
- `angular_velocity`.

# Interpretación de la pose

La pose de Turtlesim contiene:

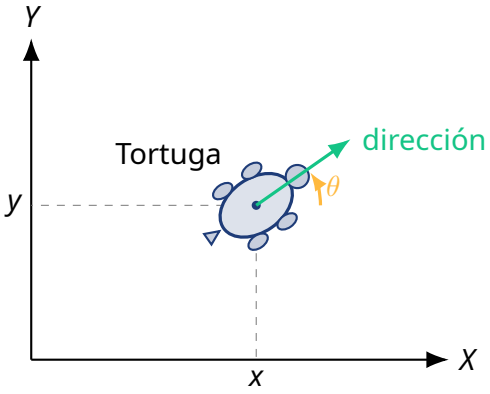
$$\begin{bmatrix} x \rightarrow \text{posición horizontal.} \\ y \rightarrow \text{posición vertical.} \\ \theta \rightarrow \text{orientación de la tortuga.} \end{bmatrix}$$

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$$

**Pose**

La pose reúne posición y orientación. En robótica móvil plana suele representarse como $(x, y, \theta)$.

# Pose y orientación




**Resultado**

El tópico `/turtle1/pose` permite observar el estado geométrico de la tortuga.

# Tipos de mensaje

Saber qué estructura viaja por un tópico

# Ver el tipo de mensaje

Para conocer el tipo de mensaje de la pose:

```
ros2 topic info /turtle1/pose
```

Una salida posible es:

```
Type: turtlesim/msg/Pose
Publisher count: 1
Subscription count: 1
```

**Resultado**

El tópico `/turtle1/pose` utiliza mensajes de tipo `turtlesim/msg/Pose`.

# Ver la estructura de un mensaje

Ejecute:

```
ros2 interface show turtlesim/msg/Pose
```

Una salida posible es:

```
float32 x
float32 y
float32 theta
float32 linear_velocity
float32 angular_velocity
```

**Idea clave**

La interfaz del mensaje indica qué datos contiene y cómo están organizados.

# Estructura de Twist

Para ver la estructura de `Twist`:

```
ros2 interface show geometry_msgs/msg/Twist
```

La estructura general es:

```
Vector3  linear
Vector3  angular
```

**Twist**

Un mensaje `Twist` representa velocidad lineal y velocidad angular.

# Laboratorio guiado

Explorar nodos, tópicos, mensajes y movimiento

# Parte 1: iniciar el sistema

Realice los siguientes pasos:

1. abra una terminal;
2. cargue ROS 2 Humble si es necesario;
3. ejecute `turtlesim_node`;
4. abra una segunda terminal;
5. ejecute `turtle_teleop_key`;
6. mueva la tortuga con el teclado.

## Actividad práctica

Observe qué ocurre cuando se presionan las flechas y relacione el movimiento con velocidad lineal y angular.

# Parte 2: inspección

Con el sistema en ejecución:

1. liste los nodos activos;
2. liste los tópicos disponibles;
3. consulte la información de `/turtle1/cmd_vel`;
4. consulte la información de `/turtle1/pose`;
5. observe la estructura de `geometry_msgs/msg/Twist`;
6. observe la estructura de `turtlesim/msg/Pose`.

**Resultado**

Esta parte permite relacionar lo que se ve en pantalla con la arquitectura interna de ROS 2.

# Parte 3: publicación manual

Publique diferentes comandos de velocidad.
Pruebe:

- avance recto;
- giro en el sitio;
- curva hacia la izquierda;
- curva hacia la derecha;
- detener la tortuga.

**Idea clave**

Cada movimiento debe interpretarse según los valores asignados a `linear.x` y `angular.z`.

# Comandos sugeridos

Avanzar recto:

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 2.0}, angular: {z: 0.0}}"
```

Girar en el sitio:

```
ros2 topic pub --once /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.0}, angular: {z: 1.5}}"
```

# Comandos sugeridos

Curva antihoraria:

```
ros2 topic pub -r 2 /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 1.0}, angular: {z: 0.7}}"
```

Curva horaria:

```
ros2 topic pub -r 2 /turtle1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 1.0}, angular: {z: -0.7}}"
```

Detenga con:

Ctrl + C

# Parte 4: observar la pose

Mientras la tortuga se mueve:

1. observe `/turtle1/pose`;
2. registre tres valores diferentes de $(x, y, \theta)$;
3. identifique cuándo cambia más x;
4. identifique cuándo cambia más y;
5. observe qué ocurre con `theta` durante un giro.

**Actividad práctica**

Relacione el valor de `theta` con la orientación geométrica de la tortuga en la ventana.

# Cierre de la sesión

## ROS 2 como infraestructura de comunicación

# ¿Qué construimos hoy?

Hoy trabajamos con:

- ROS 2 Humble;
- Turtlesim;
- nodos;
- tópicos;
- mensajes;
- publicadores;
- suscriptores;
- comandos básicos de inspección;
- mensajes `Twist`;
- pose de la tortuga.

# La idea más importante

Un robot en ROS 2 es un conjunto de nodos que intercambian mensajes

$$\text{Nodo} \rightarrow \text{Tópico} \rightarrow \text{Mensaje} \rightarrow \text{Acción}$$

**Idea clave**

Antes de programar nodos propios, es necesario comprender cómo observar, publicar e interpretar información usando la terminal.

# Preguntas de cierre

Responda:

1. ¿Qué diferencia existe entre un nodo y un tópico?
2. ¿Qué función cumple un mensaje?
3. ¿Por qué `/turtle1/cmd_vel` usa un mensaje `Twist`?
4. ¿Qué representa `linear.x`?
5. ¿Qué representa `angular.z`?
6. ¿Qué información contiene `/turtle1/pose`?

# Próxima clase

En la siguiente sesión trabajaremos con programación en Python para ROS 2.
El objetivo será construir nuestros primeros nodos usando:

`rclpy`

y desarrollar:

- un publicador de velocidad;
- un suscriptor de pose;
- un movimiento controlado de Turtlesim;
- una primera relación entre código Python y tópicos ROS 2.


def main():
print(
‘Hello,wo
if __name ==
==_main__’:
main()
¡GRACIAS POR SU ATENCIÓN!
THANK YOU!
put(‘)
Hello,wor.d’)
main